Q7 – Design a Contact-Merging Program from First Principles

Requirements and key

Merge contact sources while avoiding duplicate people. Normalise names with casefold and whitespace collapsing, and normalise phone numbers by retaining digits. A composite key (normalised name, normalised phone) identifies a likely duplicate. Do not merge solely by name because two people may share a name.

Merge policy

Keep the first record as the base and fill missing fields from later records. Conflicting non-empty values should be flagged for review rather than silently overwritten. Preserve provenance when production auditability is required.

Complexity and defence

A dictionary keyed by the composite identity gives O(n) expected processing time and O(u) space for u unique contacts. Searching a list for each incoming record can cost $O(n^2)$. The merge policy should be explicit because false merges can corrupt contact data.

Python Program

```
def normalise_name(name):
    return " ".join(name.casefold().split())

def normalise_phone(phone):
    return "".join(ch for ch in phone if ch.isdigit())

def merge_contacts(*sources):
    merged = {}
    for source in sources:
        for contact in source:
            key = (normalise_name(contact["name"]), normalise_phone(contact["phone"]))
            if key not in merged:
                merged[key] = dict(contact)
            else:
                for field, value in contact.items():
                    if value and not merged[key].get(field):
                        merged[key][field] = value
    return list(merged.values())

if __name__ == "__main__":
    a = [{"name":"Anu S","phone":"+91 98765 43210","email":""}]
    b = [{"name":" anu s ","phone":"919876543210","email":"anu@example.com"}]
    print(merge_contacts(a,b))
```